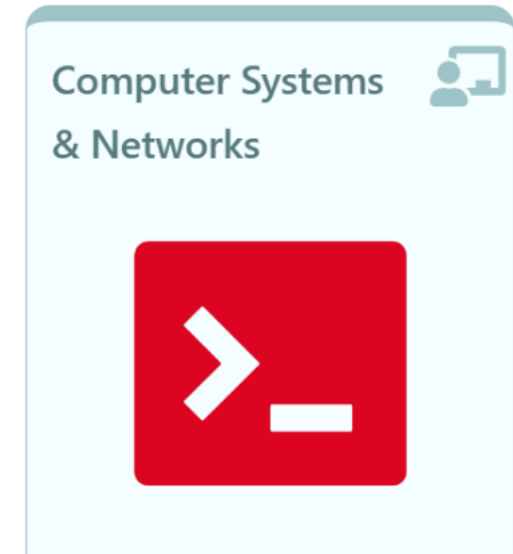
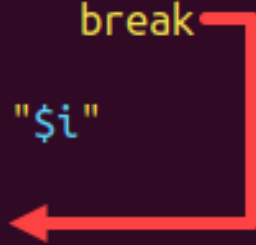


Computer Systems & Networks



```
1 #!/bin/bash
2
3 for i in {1..10}
4 do
5     if [[ $i == '2' ]]
6     then
7         echo "Number $i!"
8         break
9     fi
10    echo "$i"
11 done
12 echo "Done!"
```

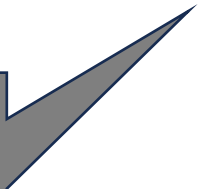


break continue

break statement

- used to **exit a loop early** before it naturally finishes.
- useful when you want to **stop iterating** based on a condition or event.

(1) The break statement **break**



Output:
Number: 0
Number: 1
All Done!

```
i=0
while [ $i -lt 5 ]
do
    echo "Number: $i"
    ((i++))
    if [[ "$i" == '2' ]]; then
        break
    fi
done
echo 'All Done!'
```

break n usage in nested loops

The **break** statement on its own will break only **the loop that it was executed in**.

break n is used to **break out of multiple nested loops** at once
break affects only for, while, or until loops — **not if statements**.

\$break n #breaks out of **n levels** of nested loops(default 1)

continue statement

continue can also be used to control the ANY loop type execution

used inside loops to **skip the rest of the current iteration** and jump to the **next iteration** of the loop

Output:

Number: 1
Skipping 2 and moving on..
Number: 3
Number: 4
All Done!

```
i=0
while [ $i -lt 5 ]
do
    ((i++))
    if [[ "$i" == '2' ]]; then
        echo "Skipping 2 and moving on.."
        continue
    fi
    echo "Number: $i"
done
echo 'All Done!'
```

```
karthick@OSTechNix:~$ /home/karthick/Documents/case.sh
```

```
Enter the first number(X) : 10
```

```
Enter the second number(Y) : 143
```

```
Addition => +
```

```
Subtract => -
```

```
Multiply => x
```

```
Division => /
```

```
Reminder => %
```

```
Choose any one opeartor : +
```

```
Addition of X and Y is 153
```

```
karthick@OSTechNix:~$ _
```

case...esac

used to execute different blocks of code depending on the **value of a variable or expression**

case...esac statement

Great alternative to nested if's

- used to simplify complex conditions
- starts with **case** statement and closes by **esac**

Useful when you wish to:

- create a menu in a script
- Have a user choose a menu option
- Have a reaction based on their choice

case...esac statement

- Always quote the variable: **case "\$var"**
- Always include a ***** to handle unexpected input
- Patterns can be:
 - Exact: **1)**
 - OR'd: **y|Y|yes|Yes)**
 - Wildcarded: ***)**

```
read word
case "$word" in
    add)
        Statement(s) to be executed if pattern1 matches
        ;;
    remove)
        Statement(s) to be executed if pattern2 matches
        ;;
    quit)
        Statement(s) to be executed if pattern3 matches
        ;;
    *)
        Default condition to be executed
        ;;
esac
```

case...esac statement

```
read userSelection
case $userSelection in
choice1)
    ACTIONS-TO-PERFORM
    ;;
choice2)
    ACTIONS-TO-PERFORM
    ;;
esac
```

case...esac statement

```
#!/bin/bash
echo "Choose a contact to display information:"
echo "[C]hris Ramsey"
echo "[J]ames Gardner"
echo "[S]arah Snyder"
echo "[R]ose Armstrong"
read person
case "$person" in
    "C" | "c" ) echo "Chris Ramsey"
echo "cramsey@email.com"
echo "27 Railroad Dr. Bayside, NY";;
    "J" | "j" ) echo "James Gardner"
echo "jgardner@email.com"
echo "31 Green Street, Green Cove Springs, FL";;
    "S" | "s") echo "Sarah Snyder"
echo "ssnyder@email.com"
echo "8059 N. Hartford Court, Syosset, NY";;
    "R" | "r") echo "Rose Armstrong"
echo "rarmstrong@email.com"
echo "49 Woodside St., Oak Forest, IL";;
    *) echo "Contact doesn't exist.";;
esac
```

Try out this weeks Lab

03: if, while, for, break & continue; case & select for menus; using sed editor

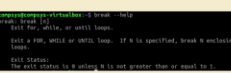
01: Conditionals and Loops



LOOPS REPEAT ACTIONS... SO YOU DON'T HAVE TO

test · if · for · while · until

02: Loop manipulation



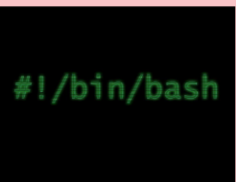
break · continue · case · select

03: sed



Stream editor · find and replace

Lab



Loops · Menus · sed

Computer System... > 03: Loops, Men... > Week 3 Class 1... > Lab

03. Menu driven scripts

case · select

case

You might want a user to confirm **yes** or **no** as an input from user

YesOrNo.sh

```
#!/bin/bash

echo -n "Do you agree with this? [y or n]: "
read word
case $word in

    [yY] | [yY][Ee][Ss] )
        echo "Agreed"
        ;;
```

select

```
root@ubuntu:~# bash select-loop.sh
What do you do?
1) Student
2) Businessman
3) Professional
4) Fresher
Enter your choice ==> 1
You're still studying
Enter your choice ==> 2
Wow you work really hard for sure!
Enter your choice ==> 3
You've been a seasoned professional
Enter your choice ==> 4
You're new in the job market
Enter your choice ==> 5
Well, you need to enter something from the list!
Enter your choice ==> █
```

select statement

- used to **create simple menus**
- Gives user a list of options to choose
- Prompt's user for an input
- **select** loops until **break** is called

```
select variable in <list>
do
    <commands>
done
```

```
options="Add Search Quit"
PS3="Choose one option: "
select name in $options; do
    if [[ "$name" == "Quit" ]]; then
        echo "Goodbyeeeee!"
        break
    elif [[ "$name" == "" ]]; then
        echo "Invalid option, try again."
    else
        echo "You selected: $name"
    fi
done
```

A few points with use of **select**

- No error checking is done, you must add this
- If the user hits **enter** without entering any data then the list of options will be displayed again.
- The loop will end when the **break** statement is issued.

```

while (testExpression) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}

do {
    // codes
    if (condition to break) {
        break;
    }
    // codes
} while (testExpression);

for (init; testExpression; update) {
    // codes
    if (condition to break) {
        break;
    }
    // codes
}

```

Summary

while do done

- Perform a set of commands while a test is true.

until do done

- Perform a set of commands until a test is true.

for do done

- Perform a set of commands for each item in a list.

break

- Exit the currently running loop.

continue

- Stop this iteration of the loop and begin the next iteration.

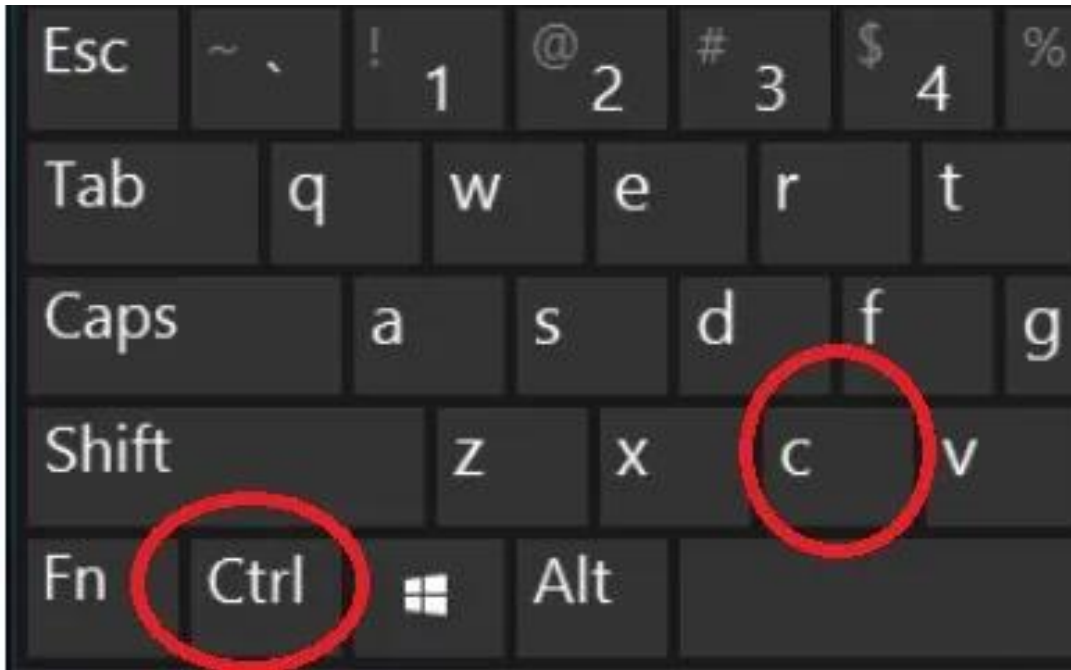
select do done

- Display a simple menu system for selecting items from a list.

case

- The case statement executes any one block of commands, based on the outcome of a pattern match.

Return back to prompt Ctrl+C



Try out this weeks Lab

03: if, while, for, break & continue; case & select for menus; using sed editor

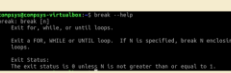
01: Conditionals and Loops



LOOPS REPEAT ACTIONS...
SO YOU DON'T HAVE TO

test · if · for · while · until

02: Loop manipulation



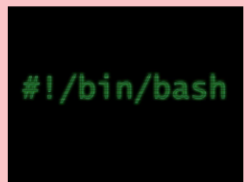
break · continue · case · select

03: sed



Stream editor · find and replace

Lab



Loops · Menus · sed

Computer Systems > 03: Loops, Menus > Week 3 Class 1... > Lab

Advancing Scripting Skills

- 01. Conditionals
- 02. Loops
- 03. Menu driven scripts**
- 04. sed
- 05. EXER Add new records
- 06. Cisco NetAcad

PRACTICE: Another menu driven script

You can create simple menus using **select** and **case**

```
#!/bin/bash
# Bash Menu Script Example

PS3='Please enter your choice: '
options=("Add" "Search" "Quit")
select opt in "${options[@]}"
do
    case $opt in
        "Add")
            echo "you chose to add a new record"
            ;;
        "Search")
            echo "you chose to search for record(s)"
            ;;
        "Quit")
            break
            ;;
        *) echo "invalid option $REPLY";;
```